11 雜湊表 解答

11-E01 負載因子與分槽

推導

$\alpha = 18/12 = 1.5$，期望成本$\Theta(1+\alpha)$。全部落入某指定槽機率12^{-18}；全部共享任一槽是十二個互斥事件的聯集，機率$12 \cdot 12^{-18} = 12^{-17}$。精確機率乘法需要鍵槽選擇獨立。

結論與檢核

失敗搜尋的預期record檢查為1.5，另有固定hash/索引開銷；不能從Theta記號讀出精確2.5時間。單次最壞仍可能掃18筆。簡單均勻假設指每鍵獨立均勻，實際鍵分布若集中，平均分析不可照搬。

11-E02 m=2^p 的 mod 碰撞

推導

0 與 8 的 mod 8 都是 0；6 與 14 的 mod 8 都是 6。二進位中，k mod 2^p 只保留最低 p 位，高位差異不影響槽。

結論與檢核

因此只要最低 3 位相同，就會碰撞。若鍵的低位元有強模式，除法法選 2 的冪可能使分布偏向少數槽。

11-E03 open addressing 的刪除

推導

查 8 先檢查槽 1，見到 NIL 就停止，會錯誤判定不存在。原因是刪除 1 破壞了 8 的探測路徑。

結論與檢核

可標記槽 1 為 DELETED 後繼續探測到槽 2，或在刪除時對後續 cluster 重插。任何方法都必須讓 search 跨過空缺。

11-E04 Universal族的有限精確機率

推導

20組(a,b)與20組有序相異(r,s)一一對應，r=b、s=a+b mod5。mod3的三個類是{0,3}、{1,4}、{2}，相異但同類的有序對為2+2+0=4組。

11 雜湊表 解答 （續）

結論與檢核

精確碰撞機率4/20=1/5<=1/3。Universality要求至多1/m，不要求剛好相等，也不要求各輸出槽邊際均勻或所有碰撞事件獨立；固定x,y必須先於選(a,b)。

11-E05 Double hashing的覆蓋週期

推導

step8有gcd(8,12)=4，週期12/4=3，probe為1,9,5,1，只能達3槽。step5與12互質，週期12，序列1,6,11,4,9,2,7,0,5,10,3,8後才回1。

結論與檢核

step8即使1,9,5都滿，也不能推論全表滿；是probe規格未覆蓋其他槽。回到起點等價12整除i×step，約去gcd後最小正i=12/gcd，給精確週期。

11-E06 Perfect hash仍要檢查原Key

推導

桶1={31}、2={20,26}、3={9,15}、4={4}，桶0/5空。次層大小1+4+4+1=10；桶2槽0為20、槽2為26，桶3槽1為9、槽3為15。26與14的第一層均2、次層均2，皆讀到26。

結論與檢核

26相等成功，14不等必回不存在。Perfect只保證集合內不同key無碰撞，沒有對所有不在K的query承諾不同位置。真檢查key才維持dictionary正確性，空桶/EMPTY也可直接拒絕。

11 雜湊表 解答 （續）

原創驗算程式附錄: hash_checks.py

```
"""Finite probability enumeration and dictionary behavior with genuine collision handling."""
from itertools import combinations, permutations, product
from fractions import Fraction
from math import gcd, log
import random

EMPTY=object()
DELETED=object()


def affine(key,a,b,p,m):
    return ((a*key+b)%p)%m


class OpenTable:
    def __init__(self,m):
        self.slots=[EMPTY]*m
    def search(self,key):
        m=len(self.slots)
        for i in range(m):
            j=(key+i)%m
            record=self.slots[j]
            if record is EMPTY:
                return None,i+1
            if record is not DELETED and record[0]==key:
                return j,i+1
        return None,m
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

11 雜湊表 解答 （續）

```
    def insert(self,key,value):
        first=None
        m=len(self.slots)
        for i in range(m):
            j=(key+i)%m
            record=self.slots[j]
            if record is not EMPTY and record is not DELETED and record[0]==key:
                self.slots[j]=(key,value)
                return j,i+1
            if record is DELETED and first is None:
                first=j
            if record is EMPTY:
                target=first if first is not None else j
                self.slots[target]=(key,value)
                return target,i+1
        if first is not None:
            self.slots[first]=(key,value)
            return first,m
        raise OverflowError
    def delete(self,key):
        index,_=self.search(key)
        if index is not None:
            self.slots[index]=DELETED
    def contents(self):
        records=[r for r in self.slots if r is not EMPTY and r is not DELETED]
        assert len(records)==len({r[0] for r in records})
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

11 雜湊表 解答 （續）

```
        return dict(records)


def perfect_build(keys,p,rng):
    keys=list(keys)
    n=len(keys)
    if not n:
        return p,0,0,[]
    while True:
        a,b=rng.randrange(1,p),rng.randrange(p)
        buckets=[[] for _ in range(n)]
        for key in keys:
            buckets[affine(key,a,b,p,n)].append(key)
        if sum(len(bucket)**2 for bucket in buckets)<4*n:
            break
    secondary=[]
    for bucket in buckets:
        size=len(bucket)**2
        if not size:
            secondary.append(None)
            continue
        while True:
            aa,bb=rng.randrange(1,p),rng.randrange(p)
            table=[None]*size
            collision=False
            for key in bucket:
                index=affine(key,aa,bb,p,size)
                if table[index] is not None:
                    collision=True
                    break
                table[index]=key
            if not collision:
                secondary.append((aa,bb,table))
                break
    return p,a,b,secondary


def perfect_contains(built,key):
    p,a,b,secondary=built
    if not secondary:
        return False
    bucket=secondary[affine(key,a,b,p,len(secondary))]
    if bucket is None:
        return False
    aa,bb,slots=bucket
    return slots[affine(key,aa,bb,p,len(slots))]==key


def main():
    families=0
    for p in [5,7,11]:
        for m in range(1,p+3):
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

11 雜湊表 解答 (續)

```
        family=[(a,b) for a in range(1,p) for b in range(p)]
        for x,y in combinations(range(p),2):
            count=sum(affine(x,a,b,p,m)==affine(y,a,b,p,m) for a,b in family)
            assert Fraction(count,len(family))<=Fraction(1,m)
        families+=1
assert sum(affine(0,a,b,5,3)==affine(1,a,b,5,3) for a in range(1,5) for b in range(5))==4
assert 13*159==2067 and (2067%256)>>4==1
for m in range(2,65):
    for step in range(1,m):
        assert len({i*step%m for i in range(m)})==m//gcd(m,step)
for w in range(1,9):
    m=2**w
    assert len({i*(i+1)//2%m for i in range(m)})==m
assert [i*i%7 for i in range(7)]==[0,1,4,2,2,4,1]
table=OpenTable(7)
for key in [1,8,15]:
    table.insert(key,str(key))
table.delete(1)
assert table.search(15)==(3,3)
assert table.insert(8,'updated')==(2,2)
assert table.insert(22,'new')==(1,4)
assert table.contents()=={22:'new',8:'updated',15:'15'}
full=OpenTable(7)
for key in range(7):
    full.insert(key,key)
for key in range(7):
    full.delete(key)
assert full.search(101)==(None,7)
assert full.insert(3,'reused')==(3,7)
rng=random.Random(20261024)
table=OpenTable(7)
reference={}
for step in range(5000):
    key=rng.randrange(21)
    operation=rng.randrange(3)
    if operation==0:
        before=table.slots[:]
        if len(reference)==7 and key not in reference:
            try:
                table.insert(key,step)
                raise AssertionError('full table must reject')
            except OverflowError:
                assert before==table.slots
        else:
            table.insert(key,step)
            reference[key]=step
    elif operation==1:
        table.delete(key)
        reference.pop(key,None)
    else:
        index,_=table.search(key)
        assert (index is not None)==(key in reference)
    assert table.contents()==reference
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

11 雜湊表 解答 （續）

```
        for query in range(21):
            index,_=table.search(query)
            assert (index is not None)==(query in reference)
    exact_probe_permutations=0
    for m in range(1,8):
        orders=list(permutations(range(m)))
        for n in range(m):
            total=0
            for order in orders:
                for index,slot in enumerate(order,1):
                    if slot>=n:
                        total+=index
                        break
                exact_probe_permutations+=1
            assert Fraction(total,len(orders))==Fraction(m+1,m-n+1)
            assert Fraction(total,len(orders))<=Fraction(m,m-n)
    m,n=4,3
    permutations4=list(permutations(range(m)))
    success_total=0
    combinations4=0
    for probes in product(permutations4,repeat=n):
        occupied=set()
        for order in probes:
            for steps,slot in enumerate(order,1):
                if slot not in occupied:
                    occupied.add(slot)
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

11 雜湊表 解答 （續）

```
                    success_total+=steps
                    break
        combinations4+=1
    success_mean=Fraction(success_total,n*combinations4)
    assert success_mean==Fraction(47,36)
    assert float(success_mean)<=log(1/(1-n/m))/(n/m)
    sets=0
    for size in range(9):
        for keys in combinations(range(8),size):
            built=perfect_build(keys,11,rng)
            assert sum(len(entry[2]) for entry in built[3] if entry is not None)<4*size if
size else built[3]==[]
            for key in range(11):
                assert perfect_contains(built,key)==(key in keys)
            sets+=1
    sample=[4,9,15,20,26,31]
    buckets=[[key for key in sample if key%6==j] for j in range(6)]
    assert buckets==[[],[31],[20,26],[9,15],[4],[]]
    assert sum(len(group)**2 for group in buckets)==10
    assert (26%6,26%4)==(14%6,14%4)==(2,2) and 26!=14
    print(f"PASS affine_families={families} open_table_mixed_operations=5000
tombstone_duplicate_and_all_deleted=True exact_failed_probe_permutations=
{exact_probe_permutations} successful_mean={success_mean} perfect_keysets={sets}
absence_key_check=True")


if __name__=='__main__':
    main()
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study